

AI Transcript

Chat 1

User:

I am doing a lab which involves building a web app (a todo list app), so I already have the functionality for adding a task, showing all tasks. The main thing about this is that it doesn't require anyone to login and it is a local app

(using react & next.js)

when it comes to editing a route, it clearly has to be dynamic such that we edit the task that the user selected, would it be smart to send every request with an id so that it will be easier for me to select that specific task from the database and edit it or is there a much more efficient and safer way

ChatGPT:

For your specific lab (a **local todo app with no authentication/users**), using the task **ID in the route** is exactly the right approach. In fact, it is the standard RESTful way to handle this.

The important distinction is: **do not send the ID everywhere unnecessarily** — use it where you are referring to a specific resource.

The usual pattern

GET /api/tasks

POST /api/tasks

GET /api/tasks/:id

PUT /api/tasks/:id

PATCH /api/tasks/:id

For editing:

/manage-task/edit/5

means:

"Open the edit page for the task whose id is 5."

Your Next.js structure would be:

app/

```
└─ manage-task/  
  └─ edit/  
    └─ [id]/  
      └─ page.js
```

The [id] is a dynamic route.

So if the user clicks:

Edit

on:

Task:

Study React

id: 7

you navigate to:

/manage-task/edit/7

Then inside page.js you get:

```
const { id } = params;
```

and fetch:

```
fetch(`/api/tasks/${id}`)
```

Your API then queries:

```
SELECT * FROM task WHERE id = ?
```

and returns that task.

Why not send the whole task object?

You might think:

"When I click edit, why don't I just send the whole task to the edit page?"

For example:

```
router.push({  
  pathname: "/edit",  
  task: selectedTask
```

}}

This is usually not ideal because:

1. **The URL does not represent the resource**

Refreshing:

/edit

loses the task information.

2. **The database is the source of truth**

The task might have changed between opening the list and clicking edit.

Example:

Task list loads:

Task id 5 → "Finish lab"

Someone changes it.

User clicks edit.

Fetching by ID gets the latest version.

3. **You avoid passing unnecessary data around**

The ID is enough information to locate the resource.

Is sending IDs unsafe?

In your lab: no.

Since:

- no login
- no users
- local database
- no sensitive data

there is no security concern.

In a real application, you would think about:

/api/tasks/5

because someone could manually change:

/api/tasks/6

and access another user's task.

But normally you solve that with authorization:

SELECT *

FROM tasks

WHERE id = ?

AND user_id = ?

The user ID comes from their authenticated session/token, not from the URL.

Your app does not have that problem.

For your lab, I would structure it like this:

Frontend:

TaskCard

|

| click Edit

↓

/manage-task/edit/[id]

Edit page:

GET /api/tasks/:id

|

↓

SQLite

|

↓

Populate form

When saving:

PUT /api/tasks/:id

|

↓

UPDATE task

SET title=?, description=?, ...

WHERE id=?

So your instinct is correct: **use IDs in dynamic routes and API endpoints**. That is not a shortcut; it is the normal design pattern. For your lab, a clean implementation would be:

/manage-task/edit/[id]

with:

GET /api/tasks/[id]

PUT /api/tasks/[id]

You are actually moving toward the same architecture you would use in a production CRUD application.

Chat 2

User:

okay I managed to implement that and how I did it in summary, I took the id from params and I was able to get the task (using one route), in the edit page what first displays is the task details with the currently stored information as it is in the database, the user is able to update the details and as they click the 'save changes' the information is updated in the database and again the information changes in the page

ChatGPT:

That implementation is actually a very solid approach for a CRUD application. What you have built is essentially the standard "**fetch → populate form → update → refresh state**" workflow.

Your flow can be summarized like this:

User clicks Edit on a task

|

↓

Navigate to /manage-task/edit/[id]

|

↓

Get id from params

|

↓

GET /api/tasks/[id]

|

↓

Query SQLite using id

|

↓

Return selected task

|

↓

Display current task details in form

|

↓

User modifies fields

|

↓

Click "Save Changes"

|

↓

PUT /api/tasks/[id]

|

↓

UPDATE task in SQLite

|

↓

Fetch updated task / update state

|

↓

Display new information

A few things you did correctly:

1. The ID is the link between the UI and database

Your edit page does not need to know the whole task initially. It only needs:

id = 5

From that, the backend knows:

SELECT *

FROM task

WHERE id = 5;

That keeps your frontend and backend nicely separated.

2. Loading the existing values first is the correct UX

This part:

"the edit page first displays the task details with the currently stored information"

is exactly how most editing interfaces work.

For example:

Title:

[Finish COMS3011A Lab]

Description:

[Implement edit route]

Due date:

[2026-08-04]

The user is not creating a new task; they are modifying an existing resource.

3. Updating the database instead of only changing the page is important

A common beginner mistake is:

`setTitle(newTitle)`

and thinking the task is edited.

But that only changes React state. Refresh the page and the change disappears.

You correctly did:

React state

↓

API request

↓

SQLite UPDATE

↓

Database permanently changed

That is the important part.

One small improvement you could consider (if you haven't already done it):

Instead of:

Save button clicked

|

↓

PUT request

|

↓

Fetch task again

|

↓

Display updated task

you can also update your local state using the response from the PUT request:

```
const response = await fetch(`/api/tasks/${id}`, {  
  method: "PUT",  
  body: JSON.stringify(updatedTask)  
});
```

```
const updated = await response.json();
```

```
setTask(updated);
```

Your API returns the updated task:

```
{  
  id: 5,  
  title: "New title",  
  description: "New description",  
  status: "Complete"  
}
```

Then React immediately updates.

However, your current method (saving then fetching again) is completely acceptable, especially for a lab. It is simple and guarantees your UI matches the database.

The only thing I would check for your lab requirements is that your edit route also handles **all editable fields**:

- title
- description
- due date
- topic

- status

and that your API validates that the task exists before updating:

UPDATE task

SET ...

WHERE id=?

If no row matches that ID, return something like:

404 Task not found

Other than that, the architecture you described is exactly what I would expect from a clean Next.js + SQLite CRUD implementation. You have basically implemented the "U" and "R" parts of CRUD properly.

Chat 3

User:

I am not sure about the design of my app, currently what I managed to do is when you visit the website, you are taken to the page when you add tasks, and there is a button 'Add Task' then there is another button below called 'Show Task'. When a user clicks show tasks they are redirected to another page where there are three buttons, 'Active Tasks', 'Archived Tasks' and 'All Tasks'. So when a user clicks 'Archived Tasks', all their archived tasks will be shown, basically showing tasks relevant to the filter or button clicked. Now I am not sure if this is a good design, I was also thinking of adding an informative message just below the buttons

ChatGPT:

Yes — I actually think your current structure is **perfectly reasonable**, especially for a university lab. I wouldn't redesign the whole thing. I would mainly improve the **navigation and visual hierarchy** so that it feels more like one cohesive todo app rather than separate pages.

Your current flow is:

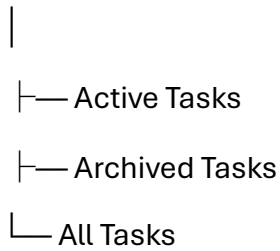
Home / Add Task

|

|— Add Task

|

|— Show Tasks



That's easy to understand. 👍

I especially like the idea of the informative message

I think that would help a lot.

For example, when the user selects **Archived Tasks**, instead of just seeing the buttons and then a list suddenly appearing, you could have:

Archived Tasks

These are tasks you have archived. Archived tasks are kept for reference and can be restored when needed.

Then the tasks underneath.

For **Active Tasks**:

Active Tasks

These are your current tasks that have not been archived.

For **All Tasks**:

All Tasks

View all your tasks, including both active and archived tasks.

That gives the user context about **what they're currently looking at**.

I would make one small design change

Instead of thinking of these as three "pages," I'd think of them as **three views of the same task-management page**.

So visually:

My Tasks

[Active] [Archived] [All]

Active Tasks

These are your current tasks
that have not been archived.

Finish COMS Lab	
Due: 10 August	
Status: In Progress	
[Edit] [Archive]	

When they click **Archived:**

My Tasks

[Active] [Archived] [All]

Archived Tasks

These tasks have been archived
and are no longer active.

Research methodology	
Due: 5 August	
Status: Complete	
[Edit] [Restore]	

That makes it immediately obvious that the **buttons are filters/views**, rather than unrelated destinations.

One other thing I'd consider

Your home page currently has:

Add Task

[Add Task]

[Show Tasks]

That's okay, but you could make the page feel more intentional:

My Todo List

Keep track of your tasks,
deadlines and progress.

[Add Task]

[View Tasks]

Then the task page handles the viewing/filtering.

I would also consider changing "**Show Task**" to "**View Tasks**" or "**View All Tasks**".

"Show Task" sounds like you're going to show one particular task, whereas you're actually taking the user to a collection of tasks.

So overall

I **wouldn't add more pages just for the sake of adding pages**. Your current architecture is simple and makes sense:

Home → Add Task / View Tasks → Active / Archived / All

I'd focus on:

- Making **View Tasks** clearer than "Show Task"

- Treating Active/Archived/All as **filters/views**
- Adding a short **heading + informative description** beneath the filter buttons
- Visually indicating which filter is currently selected
- Giving the empty state some information too

For example, if there are no archived tasks:

No archived tasks

You haven't archived any tasks yet. Tasks you archive will appear here.

That last part makes the app feel *much* more polished without making the implementation unnecessarily complicated.

Chat 4

User:

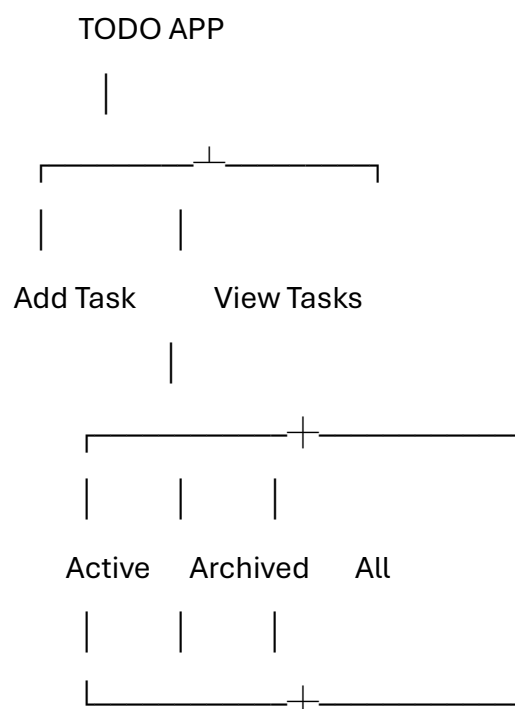
yes they are basically 3 views of the same page, not three different pages

ChatGPT:

Exactly! **That makes your design much better than it might have sounded initially.**

You're essentially building one **Task Management / Task List page**, with three different views of the same underlying data.

I'd think of your app as having two main areas:



Same Task List UI

So when the user clicks **Active**, you're not really navigating to a different "page." You're changing the **view/filter** of the task list.

I would design that page something like this

My Tasks

Manage and keep track of your tasks.

Active	Archived	All
--------	----------	-----

Active Tasks

Your current tasks are shown below.

Complete COMS Lab
Implement the edit functionality
Topic: Programming
Due: 10 August 2026
Status: In Progress
[Edit] [Archive]

Then clicking **Archived** changes only this part:

Archived Tasks

These tasks have been archived.

while the rest of the page stays essentially the same.

And this is where your informative message is a REALLY good idea

I'd make the message dynamic based on the selected view.

Selected view		Description
Active	Active Tasks	Your current tasks that have not been archived.
Archived	Archived Tasks	Tasks you've archived are kept here for reference.
All	All Tasks	View all of your active and archived tasks.

Then you can also handle the empty state:

Active with no tasks:

You don't have any active tasks yet. Add a task to get started.

Archived with no tasks:

You don't have any archived tasks yet.

All with no tasks:

You haven't created any tasks yet.

That gives the page a purpose even when there aren't any tasks to display.

One thing I'd definitely add

Make the currently selected button visually different.

For example:

[ACTIVE] [Archived] [All]



If Active is selected, it could have a different background/border, while Archived and All look like normal buttons.

Then when the user clicks Archived:

[Active] [ARCHIVED] [All]



This gives the user an immediate answer to:

"Which tasks am I currently looking at?"

And importantly, **you don't need separate components/pages for each view**. You can have one task-list component and change what it fetches/displays based on the selected filter.

So honestly, I think your basic design is good. I'd **polish the existing structure rather than redesign it**. It fits your lab requirement nicely and keeps the application easy to understand.

Chat 5

User:

I am done with most of the requirements for the lab, all functionalities are working now I moved on to testing. I decided to use Vitest where I test the database behaviour, more specifically adding tasks, archiving and the overdue calculation. But I am not quite happy with the way the tests are because it appears as though I am rewriting code (well not all) that I already wrote when I was coding functionality, is there another way to do this like maybe exporting my route functions and reusing them. My other concern is that the requirements specifically ask that a throwaway database to be use for tests so won't that mean I will have to modify my functions to take a db as one of the arguments so that when I reuse them in test I am able to use the in-memory SQLite database

ChatGPT:

This is actually a really good question because you have reached the point where **testing starts influencing the design of your code**, which is exactly what happens in real projects.

Your concern is valid: if your tests are manually recreating the same SQL queries and logic that your routes already use, then you are not really testing your application — you are testing a second implementation of it. That can lead to a situation where:

Your app code:

```
INSERT INTO tasks ...
```

Your test code:

```
INSERT INTO tasks ...
```

Both can be wrong in the same way, and your test still passes.

The better approach is exactly what you suggested: **extract reusable logic and inject the database dependency**.

1. Don't test API routes directly for database logic

A common pattern is:

Route

|

↓

Service / Database functions

|

↓

SQLite

Currently you might have something like:

app/api/tasks/route.js

POST request

|

↓

open database

|

↓

insert task

Instead, separate it:

lib/

└─ db.js

└─ taskService.js

app/api/tasks/route.js

taskService.js

Example:

```

export function createTask(db, task) {
  const stmt = db.prepare(`
    INSERT INTO task
    (title, description, due_date, topic, status)
    VALUES (?, ?, ?, ?, ?)
  `);

  const result = stmt.run(
    task.title,
    task.description,
    task.due_date,
    task.topic,
    task.status
  );

  return result.lastInsertRowid;
}

```

Now this function does not care where the database came from.

It just receives:

db

and performs the operation.

2. Your API route becomes very thin

Now your route does:

```

import { createTask } from "@lib/taskService";
import db from "@lib/db";

```

```

export async function POST(request) {

```

```
const task = await request.json();

const id = createTask(db, task);

return Response.json({ id });
}
```

The route's job is only:

- receive HTTP request
 - validate input
 - call business logic
 - return response
-

3. Your test uses a throwaway database

This is where dependency injection helps.

Your test can create:

```
import Database from "better-sqlite3";
```

```
const testDb = new Database(":memory:");
```

Then:

```
createTask(testDb, {
  title: "Test task",
  description: "Testing",
  due_date: "2026-08-12",
  topic: "Lab",
  status: "Todo"
});
```

Now your test database exists only during the test.

Your real database:

todo.db

is untouched.

Your test database:

:memory:

is destroyed after the test.

4. This also solves your concern about modifying functions

You don't have to make your existing route functions accept a database.

Instead, move the database-dependent parts into functions that accept a database.

Think:

Before:

POST route

└— open database

└— insert task

└— calculate stuff

└— return response

After:

POST route

|

↓

createTask(db, task)

createTask()

|

↓

SQLite

The route still uses your normal database:

```
createTask(realDb, task)
```

The test uses:

```
createTask(testDb, task)
```

Same function. Different database.

5. For your specific tests

Your planned tests are actually good:

Adding tasks

Test:

Given empty database

When createTask() is called

Then task exists in database

Archiving

Test:

Given active task

When archiveTask(id)

Then archived = 1

Overdue calculation

This one depends on where your logic is.

If you have something like:

```
if (due_date < today)
```

```
    overdue = true
```

extract that:

```
export function isOverdue(dueDate) {  
    return new Date(dueDate) < new Date();  
}
```

Then test:

```
expect(isOverdue("2020-01-01")).toBe(true);  
expect(isOverdue("2030-01-01")).toBe(false);  
No database needed.
```

One thing I would avoid

Don't export your route handlers and call:

```
POST(request)
```

inside your tests.

You *can* do that, but then you are testing:

- HTTP objects
- Next.js behaviour
- request parsing
- responses

instead of your actual task logic.

For a lab, testing the underlying functions is cleaner.

So yes, your instinct is correct:

"won't that mean I will have to modify my functions to take a db as one of the arguments?"

Yes — and that is actually a good design improvement, not a workaround.

You are basically applying **dependency injection**, which is exactly the pattern used to make database code testable.

For your lab I would probably refactor into:

lib/

```
└─ database.js    // creates real db connection
```

```
└─ taskOperations.js // createTask, updateTask, archiveTask, getTasks
```

app/api/

└─ tasks/

└─ route.js // calls taskOperations

Then Vitest only tests taskOperations.js using :memory: SQLite. This will make your tests shorter, cleaner, and much more meaningful.

Chat 6

User:

```
import {describe, test, expect} from "vitest";
import Database from "better-sqlite3";

function createThrowAwayDatabase(){
  const db = new Database(":memory:");

  db.exec(CREATE TABLE task( id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT
  NOT NULL, description TEXT, due_date TEXT NOT NULL, topic TEXT NOT NULL, status
  TEXT NOT NULL, archived INTEGER NOT NULL DEFAULT 0 ); );

  return db;
}

describe("Task database behaviour", ()=>{
  test("creates a task with all required fields", ()=>{
    const db = createThrowAwayDatabase();

    db.prepare(`INSERT INTO task (title,description,due_date,topic,status) VALUES
    (?, ?, ?, ?, ?)`).run("Study COMS3011A", "Finish testing", "2026-08-20", "University",
    "ToDo");

    const task = db.prepare("SELECT * FROM task WHERE title = ?").get("Study
    COMS3011A");

    expect(task.title).toBe("Study COMS3011A");
    expect(task.description).toBe("Finish testing");
    expect(task.due_date).toBe("2026-08-20");
    expect(task.topic).toBe("University");
    expect(task.status).toBe("ToDo");
```



```

expect(task.archived).toBe(0);

db.close();

})

test("archives a task without deleting it", ()=>{
  const db = createThrowAwayDatabase();

  db.prepare(`INSERT INTO task (title, description, due_date, topic, status) VALUES
  (?, ?, ?, ?, ?)`).run("Archived Test", "Testing archive", "2026-08-11", "Testing", "ToDo");

  const task = db.prepare("SELECT *FROM task WHERE title = ?").get("Archived Test");

  db.prepare("UPDATE task SET archived = 1 WHERE id = ?").run(task.id);

  const archivedTask = db.prepare("SELECT * FROM task WHERE id = ?").get(task.id);

  expect(archivedTask).toBeDefined();
  expect(archivedTask.archived).toBe(1);

  db.close();

})

test("updates an existing task", ()=> {
  const db = createThrowAwayDatabase();

  db.prepare(`
    INSERT INTO task
      (title, description, due_date, topic, status)
      VALUES (?, ?, ?, ?, ?)
  `).run(
    "Old Title",
    "Old description",

```

```
"2026-08-20",  
"University",  
"ToDo"  
);
```

```
const task = db.prepare(  
  "SELECT * FROM task WHERE title = ?"  
).get("Old Title");
```

```
db.prepare(`  
  UPDATE task  
  SET title = ?, status = ?  
  WHERE id = ?  
`).run(  
  "Updated Title",  
  "Complete",  
  task.id  
);
```

```
const updatedTask = db.prepare(  
  "SELECT * FROM task WHERE id = ?"  
).get(task.id);
```

```
expect(updatedTask.title).toBe("Updated Title");
```

```
expect(updatedTask.status).toBe("Complete");
```

```
db.close();
```

```
});
```

```
test("identifies overdue tasks", () => {
```

```
  const db = createThrowAwayDatabase();
```

```
  db.prepare(`
```

```
    INSERT INTO task
```

```
    (title, description, due_date, topic, status)
```

```
    VALUES (?, ?, ?, ?, ?)
```

```
  `).run(
```

```
    "Overdue Task",
```

```
    "Past deadline",
```

```
    "2026-08-01",
```

```
    "University",
```

```
    "ToDo"
```

```
  );
```

```
  const task = db.prepare(
```

```
    "SELECT * FROM task WHERE title = ?"
```

```
  ).get("Overdue Task");
```

```
  const today = new Date("2026-08-11");
```

```
  const dueDate = new Date(task.due_date);
```

```
const days = (dueDate - today) / 86400000;
```

```
expect(days).toBeLessThan(0);
```

```
db.close();
```

```
});
```

```
})
```

this is how my tests file currently looks like (please don't provide code or modify my code rather explain)

so the code is exactly the same as my implementation so wouldn't that mean if my test fail then my implementation fails thus testing my project

ChatGPT:

Yes — **that is the exact problem with your current tests.** You have identified an important testing issue.

Right now, your tests are essentially doing this:

Your application

```
|
|— INSERT task
|— UPDATE task
└─ calculate overdue
```

Your test

```
|
|— INSERT task
|— UPDATE task
└─ calculate overdue
```

The test isn't actually exercising the code that your application uses. It's performing the same operations independently.

So what happens if your implementation is wrong?

Suppose your actual application has this bug:

The archive functionality accidentally deletes the task instead of setting archived = 1.

Your current test doesn't call your archive functionality. It does its own:

UPDATE task SET archived = 1...

So the test can **pass even though your application is broken**.

That's the key issue.

Your tests are currently testing SQLite, not your application

Take your first test.

Your application probably has something like:

User clicks Add Task

↓

POST /api/tasks

↓

your implementation

↓

INSERT INTO task ...

But your test does:

Test

↓

INSERT INTO task ...

↓

SELECT FROM task

↓

assertions

You're basically asking:

"Does SQLite correctly store the values that I just told SQLite to store?"

That's not particularly useful for testing *your* implementation.

The same applies to your archive test.

You're manually doing the archive operation in the test, rather than asking **your application's archive functionality** to do it.

What you actually want

You want the relationship to look like:

YOUR IMPLEMENTATION

|

↓

createTask(db)

|

↓

test database

|

↓

SQLite

And your test should be:

"Call the exact function my application uses."

↓

"Give it a throwaway database."

↓

"Check whether it did the correct thing."

That way, if your implementation contains a bug, the test can actually catch it.

And this is where your `:memory:` database comes in

Your concern from your previous message was:

"Won't I have to modify my functions to take a db?"

Yes, and that's the point.

You aren't changing your application's *behaviour*. You're changing its **structure so that the database dependency can be substituted**.

Think about your real application:

```
createTask(realDatabase, task)
```

Your test:

```
createTask(testDatabase, task)
```

The function itself is identical.

The only difference is **which database it operates on**.

So your production application continues using:

```
todo.db
```

while Vitest uses:

```
:memory:
```

This is called **dependency injection**.

Your tests can then become much more meaningful

For example, your archive test should conceptually be:

1. Create throwaway database
2. Put a task into it
3. Call YOUR archive function
4. Look at the database
5. Check:

- task still exists
- archived = 1

Notice the important difference:

Current test

Test → UPDATE task SET archived = 1

Better test

Test → YOUR archive function → UPDATE task SET archived = 1

That middle part is what you're currently missing.

Your overdue test has an additional problem

This one is even more obvious.

You're testing:

`dueDate - today < 0`

But if that's exactly the calculation your application performs, you're simply reproducing the calculation inside your test.

For example, imagine your implementation accidentally has:

`days = (today - dueDate) / 86400000`

instead of:

`days = (dueDate - today) / 86400000`

Your test could still pass because you've independently written the same calculation you're trying to verify.

A better test would exercise your application's **overdue calculation function** and then verify its result.

So should you throw away your current tests?

No!

They actually give you a useful starting point.

Your database setup is good:

`createThrowAwayDatabase()`

is exactly the kind of thing you want.

Your assertions are also useful.

What's missing is the connection between:

test → your application code → test database

rather than:

test → database directly

Think of testing as this question

A good test asks:

"If I give my application X, does my application produce Y?"

Your current test asks:

"If I manually perform X using SQLite, does SQLite contain Y?"

That's why you had that feeling of:

"I'm just rewriting my code."

Your feeling was completely correct. You're essentially writing a second implementation of your functionality inside the test.

And this is actually a good point to refactor your application

Since you've already finished the functionality, I wouldn't rewrite everything.

You can identify the pieces of your routes that actually perform the database/business operations:

- create task
- archive task
- update task
- calculate overdue

and extract those into reusable functions.

Then:

API route

↓

your reusable function

↓

database

and:

Vitest

↓

same reusable function

↓

throwaway database

Now your tests are genuinely testing **your project**, rather than testing whether SQLite does what you tell it to do.

And importantly, **you don't need to test every single line of your Next.js route**. Your lab's database-behaviour tests can focus on the actual operations, while the route itself can remain responsible for HTTP/request/response handling.

Chat 7

User:

okay that's understandable

ChatGPT:

Great. And honestly, noticing this issue before submitting is a really good sign — it means you're thinking about testing in the right way rather than just trying to make the test coverage number look good.

Your current tests are not "bad"; they are just testing at the wrong level.

The way you originally approached it is a very common first attempt:

1. Create a temporary database ✓
2. Perform the database operation ✓
3. Check the result ✓

The missing piece is that step 2 should not be the test rewriting the application behaviour. It should be the test **calling the application behaviour**.

A good mental model for your lab is:

Production

User action



Next.js route



Task operation function



SQLite database (todo.db)

Testing

Vitest



Task operation function



SQLite database (:memory:)

The route and the test both use the same underlying logic.

For your lab specifically, I would probably not over-engineer it. You don't need a big architecture with repositories, classes, dependency containers, etc. A simple separation is enough:

- **Routes** → handle HTTP things:
 - receiving requests
 - extracting parameters
 - returning responses
- **Task functions** → handle application behaviour:

- create task
- update task
- archive task
- retrieve tasks
- overdue calculation
- **Database setup** → creates either:
 - your real database
 - your throwaway test database

That would make your code cleaner and your tests much easier.

Also, one thing worth mentioning: because this is a **local todo app**, your testing choices are actually quite appropriate. You don't need to worry about mocking an external database, users, authentication, etc. Testing the actual SQLite behaviour with `:memory:` is a good choice because it verifies that your SQL and database interactions work.

So your next step is probably not "write more tests", but rather:

1. Identify repeated database logic inside your routes.
2. Move that logic into reusable functions.
3. Make those functions accept a database connection.
4. Let both your app and Vitest provide the database they want.

Then your existing test structure will become much shorter and much more meaningful.